Anthony Okolie

Portfolio Assistant Knowledge Base

Source document for retrieval-augmented generation (RAG)

Document Purpose

This document is a source of truth for Anthony Okolie's portfolio assistant. It contains biographical, educational, professional, technical, and project information intended to support retrieval-augmented generation (RAG). Answers about Anthony must be grounded in the facts in this document. Do not infer unlisted qualifications, employment, project outcomes, or deployment status.

The portfolio assistant should distinguish academic projects, personal projects, professional employment, and learning or exposure. When a detail is not documented here, it should say that the available information does not establish the answer.

Profile

Name: Anthony Okolie. Location: Winnipeg, Manitoba, Canada. Portfolio: https://anthonyokolieportfolio.netlify.app (also listed as anthonyokolieportfolio.netlify.app). GitHub: https://github.com/AnOkolie. LinkedIn: https://linkedin.com/in/AnOkolie. Email: Anthonyokolie.ace@gmail.com. Phone: +1 (431) 278-1445.

Anthony is a Computer Science Honours graduate from the University of Manitoba with hands-on project experience in full-stack application development, backend services, REST APIs, database-backed systems, real-time applications, data pipelines, and AI-enabled workflows. His documented technologies include Java, C#, TypeScript, JavaScript, Python, SQL, React, Node.js, Express, Spring Boot, ASP.NET Core, PostgreSQL, MySQL, MongoDB, Docker, and AWS.

Career interests include product-focused software development, backend and full-stack engineering, and emerging AI capabilities that improve workflows and decision-making.

Education

University of Manitoba, Winnipeg, Manitoba. Bachelor of Computer Science (Honours), September 2021 to June 2026. Graduated from the Honours program. Dean's Honour List (2021).

Relevant coursework: Data Structures and Algorithms; Operating Systems; Database Systems; Software Engineering; Object-Oriented Programming; Data Mining.

Technical Skills

Programming languages: Java, C#, TypeScript, JavaScript, Python, SQL, Go, Rust.

Frontend: React, Angular, HTML5, CSS3.

Backend and frameworks: Node.js, Express, Spring Boot, ASP.NET Core, FastAPI, REST APIs, WebSockets, SignalR, JWT authentication.

Databases and data: PostgreSQL, MySQL, MongoDB, Redis, Prisma ORM, Entity Framework Core, SQLAlchemy, Alembic.

AI and LLM development: OpenAI API, LLM-powered classification and summarization, embeddings, vector search, retrieval-augmented generation (RAG), LlamaIndex.TS, Qdrant, AI-assisted development with Claude Code.

Cloud, infrastructure, and developer tools: Docker, Git, GitHub, CI/CD, AWS (Elastic Beanstalk, RDS), Azure (basic), Vercel, Railway, Sevalla, Qdrant Cloud.

Testing and quality: JUnit, Vitest, Cypress, Playwright, xUnit integration testing.

Networking and security: Linux, TCP/IP, DNS, HTTP/HTTPS, Wireshark, authentication, authorization, BCrypt password hashing, JWT, refresh-token rotation, API rate limiting, encryption, audit logging.

Other tools: Excel (data analysis and structured datasets), WordPress, Rank Math, Google Search Console, RapidMiner.

Professional Experience

DNOJ Accounting and Tax | Web Developer / Data Assistant | May 2025 September 2025

Improved WordPress SEO by connecting Google Search Console, comparing SEO tools based on features and cost, and configuring Rank Math.

Contributed to a 10% increase in website traffic by restructuring page headers and above-the-fold content around researched search terms.

Additional resume material describes supporting reporting and dashboard deployments, cleaning and validating structured financial datasets, documenting data workflows, and communicating technical findings to non-technical stakeholders.

Canada Revenue Agency | Tax Processing Assistant, T1 Matching Assessor | May 2022 December 2022

Processed more than 1,000 tax cases accurately and confidentially by validating taxpayer documentation against processing requirements.

Investigated missing or inconsistent information, resolved documentation and financial discrepancies, and followed up with clients to request supporting information.

Worked in a high-volume queue, prioritized cases to meet deadlines, and documented decisions while following established procedures.

Projects

Portfolio Chatbot and Retrieval-Augmented Generation (RAG) System | In development

Technologies: JavaScript, Node.js, Express, LlamaIndex.TS, OpenAI API, text-embedding-3-small, gpt-4o-mini, Qdrant, Docker.

Anthony is building a portfolio chatbot that answers visitor questions using information about his background, skills, experience, and projects. The architecture separates a browser-based frontend from a Node.js/Express backend. The backend embeds a visitor's question, retrieves relevant portfolio-document chunks from Qdrant using vector similarity search, and supplies the retrieved context to an OpenAI chat model to generate a grounded response.

The document-ingestion workflow loads source documents, splits them into chunks, generates embeddings, and stores vectors with text payloads in a Qdrant collection named PORTFOLIO_CHATBOT. The current embedding model is OpenAI text-embedding-3-small, using its default 1536-dimensional output. The collection is configured for cosine distance. The retrieval flow queries Qdrant for the most relevant chunks, extracts their text payloads, and passes the context and user question to the chat model.

The project uses LlamaIndex.TS for document and embedding workflow support, the OpenAI API for embeddings and chat completion, and Qdrant as the vector database. The deployment plan discussed is Cloudflare Pages for the static frontend, Render for the Express backend, and Qdrant Cloud for hosted vector storage. Deployment should be described as planned unless confirmed complete.

The chatbot's intended answer policy is to answer questions about Anthony using retrieved source context, avoid inventing skills or professional experience, and acknowledge when the retrieved information does not establish an answer. The project is ongoing; do not claim production deployment, measured retrieval accuracy, user counts, or evaluation results unless later documented.

Ride-Sharing Platform | C#, ASP.NET Core, Entity Framework Core, MySQL, React, TypeScript

Built a full-stack ride-hailing application with rider and driver roles, vehicles, trips, simulated payments, and ratings. Implemented authentication using short-lived JWT access tokens, HttpOnly refresh cookies with rotation and reuse detection, BCrypt password hashing, role-based access, and per-IP rate limiting.

Added real-time dispatch and live trip tracking with SignalR (WebSockets), offering trips to the nearest eligible driver with timed offers. Used concurrency tokens to resolve accept/cancel races. Validated AI-generated code with xUnit integration tests covering authentication, profiles, matching, and payments, including tests against MySQL to identify issues not caught by the in-memory provider. Claude Code was used during development for code generation, debugging, and testing.

AI Document Processing and Business Automation | Python, FastAPI, PostgreSQL, Docker

Built a six-stage document-processing workflow that turns synthetic invoices into structured records through ingestion, extraction, LLM classification, validation, persistence, and business-rule processing. Developed six or more authenticated REST endpoints for document upload, processing status, review decisions, retries, and audit-history retrieval. Used SQLAlchemy and Alembic to manage workflow data.

Implemented more than ten deterministic validation checks, including required fields, arithmetic totals, purchase-order matching, duplicate detection, and idempotency. Exceptions are routed to manual review. Used AI-assisted development to generate, debug, and refine code, reviewing changes against requirements and testing the complete workflow.

Real-Time Collaboration Board | Node.js, Express, React, TypeScript, PostgreSQL, Redis, Prisma, Docker

Built a collaborative web application that allows multiple users to edit shared content simultaneously. Developed React and TypeScript frontend components and Node.js/Express APIs for application logic, authentication, and persistent data management. Used WebSockets for concurrent client communication and synchronized application state, Redis Pub/Sub and caching for active users and real-time session state, and PostgreSQL schemas accessed through Prisma ORM. Containerized services with Docker and deployed backend infrastructure to cloud platforms.

Repository: https://github.com/AnOkolie/collabBoard. Live demo: https://collab-board-pink-eight.vercel.app/.

Trade Execution and Risk Monitoring Microservice | Spring Boot, TypeScript, PostgreSQL, Docker

Designed and refactored backend workflows for structured trade processing. Designed PostgreSQL schemas and optimized queries, reporting an approximately 30% improvement in request-processing performance. Built workflows to process concurrent trade requests while maintaining validation and execution consistency. Separated validation, execution, and persistence layers to improve maintainability, and containerized application components with Docker.

Repository: https://github.com/AnOkolie/TradeExecutionMicroservice.

Real-Time Chat Application | Node.js, React, WebSockets, MongoDB, Docker

Built a real-time messaging platform supporting one-to-one and group communication between concurrent users. Implemented WebSocket-based message delivery, chat rooms, and user activity tracking. Designed REST APIs for authentication, user management, and chat-room creation, with MongoDB for persistent storage. Containerized and deployed the application on Sevalla using Docker; supported testing with 10 beta users. Investigated and resolved synchronization and state-management issues.

Repository: https://github.com/AnOkolie/chat-app.

ETL and Data Mining Pipeline (Coursework) | Python, RapidMiner, SQL

Built Python preprocessing scripts to clean, validate, and standardize raw datasets for downstream analysis. Designed SQL transformations for filtering, aggregation, and structured data preparation. Developed RapidMiner workflows for visual ETL, feature selection, pattern discovery, and dataset transformation. Combined code-based Python/SQL processing with visual data-mining workflows to produce analysis-ready datasets.

Repository: https://github.com/AnOkolie/data-pipeline.

News Intelligence Pipeline | JavaScript, Playwright, OpenAI, PostgreSQL

Built an AI-powered pipeline to scrape, classify, and summarize Hacker News content using Playwright, OpenAI, and PostgreSQL. Automated ingestion and processing of more than 100 articles per run, generating structured metadata including sentiment, topics, and keyword analysis. Designed PostgreSQL-backed workflows for storing, querying, and analyzing processed article data. Developed a CLI analytics dashboard for trending topics, sentiment breakdowns, and feed-level insights. Added validation and error-handling workflows.

Repository: https://github.com/AnOkolie/news-scrapper.

Secure Secret Vault API | Go, PostgreSQL, Docker, REST APIs

Built a Go backend service for secure storage and retrieval of encrypted user secrets through authenticated REST APIs. Implemented JWT-based authentication, request validation, and encryption workflows to protect sensitive data before persistence. Designed relational schemas and backend services for secret lifecycle management, access control, and audit logging. Containerized the application with Docker and developed automated testing workflows. Added structured logging and error handling.

Repository: https://github.com/AnOkolie/secret-vault.

Additional Exposure And Development Practices

Active Directory and remote connectivity/support tools: exposure gained while shadowing a government IT professional. This should be described as shadowing or observational exposure, not independent professional system administration.

AI-assisted development: familiar with Claude Code for code generation, debugging, testing, and reviewing generated changes against project requirements. This is a development aid and should not be represented as professional AI model training experience.

Across projects, documented practices include API design, database schema design, authentication and authorization, real-time communication, concurrency handling, containerization, automated testing, validation, error handling, and technical documentation.

Retrieval And Answering Guidance

Use the exact project names and technology stacks listed above. Related aliases include: portfolio chatbot = RAG chatbot = retrieval-augmented generation project; collaboration board = real-time collaborative application; trade execution microservice = trade-processing backend; news intelligence pipeline = Hacker News scraper and summarizer; secret vault = encrypted secrets API.

When asked about the RAG project, explain the pipeline in order: source documents; chunking; OpenAI embeddings; Qdrant vector storage; query embedding; similarity retrieval; retrieved text context; OpenAI chat completion; response grounded in context.

Do not say Anthony trained or fine-tuned an LLM. The documented project uses a pretrained OpenAI chat model with retrieval-augmented generation. Do not conflate embeddings with model training. Do not claim the chatbot is deployed or production-ready unless an updated source confirms that.

Do not describe shadowing exposure to Active Directory as administering enterprise infrastructure. Do not invent employers, dates, project metrics, team sizes, users, awards, certifications, or responsibilities.

If two source documents disagree, prefer the more recent resume entry where a clear update is apparent. For DNOJ, the newer resume lists MaySeptember 2025 and the title Web Developer / Data Assistant; an older resume lists MayJuly 2025 and Web Designer / Data Assistant. If asked about the discrepancy, state that the source resumes differ rather than pretending both are identical.

If the source does not answer a question, say that the information is not currently documented. Keep responses concise, conversational, and factual.